

Setor de Educação Profissional e Tecnológica

Tecnologia em Análise e Desenvolvimento de Sistemas

DS141 - Estruturas de Dados II

10 - Tabelas Hash

Prof. Alex Kutzke

Julho de 2021

Tradução adaptada dos slides presentes em [1]. Imagens e exemplos também retirados do mesmo material.

[1] <<https://algs4.cs.princeton.edu/lectures/keynote/34HashTables.pdf>>

Roteiro

Motivação

Funções Hash

Encadeamento Externo (Separate Chaining)

Encadeamento Interno (Linear Probing)

Motivação

Resumo Implementações Tabela de Símbolos

implementation	guarantee			average case			ordered ops?	key interface
	search	insert	delete	search hit	insert	delete		
sequential search (unordered list)	N	N	N	$\frac{1}{2} N$	N	$\frac{1}{2} N$		<code>equals()</code>
binary search (ordered array)	$\lg N$	N	N	$\lg N$	$\frac{1}{2} N$	$\frac{1}{2} N$	✓	<code>compareTo()</code>
BST	N	N	N	$1.39 \lg N$	$1.39 \lg N$	$\sqrt{N}$	✓	<code>compareTo()</code>
2-3 tree	$c \lg N$	$c \lg N$	$c \lg N$	$c \lg N$	$c \lg N$	$c \lg N$	✓	<code>compareTo()</code>
red-black BST	$2 \lg N$	$2 \lg N$	$2 \lg N$	$1.0 \lg N^*$	$1.0 \lg N^*$	$1.0 \lg N^*$	✓	<code>compareTo()</code>

Fonte: (SEDGWICK; WAYNE, 2014)

Dá pra fazer melhor?

Resumo Implementações Tabela de Símbolos

implementation	guarantee			average case			ordered ops?	key interface
	search	insert	delete	search hit	insert	delete		
sequential search (unordered list)	N	N	N	$\frac{1}{2} N$	N	$\frac{1}{2} N$		equals()
binary search (ordered array)	$\lg N$	N	N	$\lg N$	$\frac{1}{2} N$	$\frac{1}{2} N$	✓	compareTo()
BST	N	N	N	$1.39 \lg N$	$1.39 \lg N$	$\sqrt{N}$	✓	compareTo()
2-3 tree	$c \lg N$	$c \lg N$	$c \lg N$	$c \lg N$	$c \lg N$	$c \lg N$	✓	compareTo()
red-black BST	$2 \lg N$	$2 \lg N$	$2 \lg N$	$1.0 \lg N^*$	$1.0 \lg N^*$	$1.0 \lg N^*$	✓	compareTo()

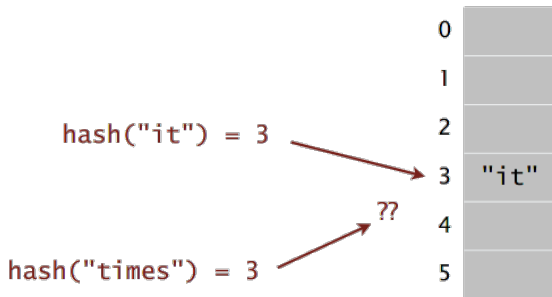
Fonte: (SEDGEWICK; WAYNE, 2014)

Dá pra fazer melhor? **Sim!** Mas com um acesso diferente aos dados.

Tabelas Hash: estratégia básica

Armazenar os itens em uma tabela indexada (o índice é uma função da chave armazenada).

Função Hash: Método para computar o índice de um array a partir de uma chave.



Fonte: (SEdgeWICK; WAYNE, 2014)

Tabelas Hash: estratégia básica

Questões importantes:

- Computar a função hash;
- Teste de igualdade: método para checar se duas chaves são iguais;
- Resolução de colisão: algoritmo e estruturas de dados para resolver o caso de quando duas ou mais chaves são alocadas em um mesmo índice do array.

Relação clássica Armazenamento X Tempo

- Sem limitação de espaço: função hash é trivial;
- Sem limitação de tempo: resolução de colisão é trivial (busca sequencial simples);
- Limitações de tempo e de espaço: hashing no mundo real. :)

Motivação
○○○○

Funções Hash
●○○○○○○○○○○○○○○○○

Encadeamento Externo (Separate Chaining)
○○○○○○○○○

Encadeamento Interno (Linear Probing)
○○○○○○○○○○○○○○○○

Funções Hash

Funções Hash

Objetivo ideal: Organizar as chaves uniformemente para produzir um índice.

- Eficientemente computável;
- Cada índice tem a mesma probabilidade de receber chaves.

Exemplo: Números de telefone (9 9999 9999).

- Ruim: primeiros três dígitos;
- Melhor: últimos três dígitos;

Desafio prático: Cada tipo de chave necessita de uma abordagem diferente.

Criando funções Hash

- Algumas linguagens de programação já implementam funções hash para alguns tipos básicos:
 - Java: `.hashCode()`
- Diferentes estratégias podem ser utilizadas, dependendo do tipo de dado;
- Método mais comum: **hash modular**.

Hash Modular

Dado um array (tabela) com M posições, o **Hash Modular** se baseia no cálculo do módulo entre o valor da chave a ser armazenada e M :

$$\text{hash}(x) = x \bmod M$$

Por exemplo, para $x = 98$ e $M = 10$:

$$\text{hash}(x) = 98 \bmod 10 = 8$$

Assim, a chave 98 seria armazenada na posição 8 do array.

Valor de M: A escolha do tamanho do array tem um papel importante; Vejamos alguns exemplos.

Exemplo 1 Função Hash

Hash modular com chaves distribuídas uniformemente

$$h(x) = x \% 12$$

Chaves a serem armazenadas:

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11,
12, 13, 14, 15, 16, 17, 18, 19,
20, 21, 22, 23.

Hashcode	Chave Armazenada
0	12, 24
1	1, 13
2	2, 14
3	3, 15
4	4, 16
5	5, 17
6	6, 18
7	7, 19
8	8, 20
9	9, 21
10	10, 22
11	11, 23

Exemplo 2 Função Hash

Hash modular com chaves NÃO distribuídas uniformemente

$$h(x) = x \% 12$$

Chaves a serem armazenadas:
3, 6, 9, 12, 15, 18, 21, 24, 27,
30, 33, 36, 39, 42, 45.

Hashcode	Chave Armazenada
0	12, 24, 36, 48
1	—
2	—
3	3, 15, 27, 39
4	—
5	—
6	6, 18, 30, 42
7	—
8	—
9	9, 21, 33, 45
10	—
11	—

Como melhorar?

Exemplo 2 Função Hash

Hash modular com chaves NÃO distribuídas uniformemente

$$h(x) = x \% 12$$

Chaves a serem armazenadas:
3, 6, 9, 12, 15, 18, 21, 24, 27,
30, 33, 36, 39, 42, 45.

Hashcode	Chave Armazenada
0	12, 24, 36, 48
1	—
2	—
3	3, 15, 27, 39
4	—
5	—
6	6, 18, 30, 42
7	—
8	—
9	9, 21, 33, 45
10	—
11	—

Como melhorar? Utilizar um valor primo para M

Exemplo 3 Função Hash

Hash modular com chaves NÃO distribuídas uniformemente e M primo

$$h(x) = x \% 11$$

Chaves a serem armazenadas:
3, 6, 9, 12, 15, 18, 21, 24, 27,
30, 33, 36, 39, 42, 45.

Hashcode	Chave Armazenada
0	33
1	12, 45
2	24
3	3, 36
4	15
5	27
6	6, 39
7	18
8	30
9	9, 42
10	21

Exemplo 4 Função Hash

Hash modular para char's com chaves NÃO distribuídas uniformemente

$$h(x) = x \% 9$$

Chaves a serem armazenadas:

c, f, i, l, o, r, u, x.

Char	Código ASCII	% 9	% 7
c	99	0	1
f	102	3	4
i	105	6	0
l	108	0	3
o	111	3	6
r	114	6	2
y	117	0	5
x	120	3	1

Hashcode	Chave Armazenada
0	c, l, u
1	—
2	—
3	f, o, x
4	—
5	—
6	i, r
7	—
8	—

Exemplo 5 Função Hash

Hash modular para char's com chaves NÃO distribuídas uniformemente e M primo

$$h(x) = x \% 7$$

Chaves a serem armazenadas:

c, f, i, l, o, r, u, x.

Char	Código ASCII	% 9	% 7
c	99	0	1
f	102	3	4
i	105	6	0
l	108	0	3
o	111	3	6
r	114	6	2
y	117	0	5
x	120	3	1

Hashcode	Chave Armazenada
0	i
1	c, x
2	r
3	l
4	f
5	u
6	o

Chaves complexas

O hash modular pode ser utilizado para tipos simples de dados, mas também **para tipos complexos**;

- Estratégia comum: separar a chave em partes, realizar o hash modular em cada parte e depois combinar os hashes obtidos em um único;
- Teoria semelhante à conversão de números para bases diferentes:
 - Por exemplo, de binário para decimal;
 - Cada dígito do número binário seria “uma parte” da chave:

$$10101_2 = 1 * 2^4 + 0 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 * 2^0 = 21_{10}$$

Exemplo: hash para strings

Para strings, podemos considerar cada carácter como “uma parte” do string. Considere uma string s de tamanho n e uma tabela de m posições (KOGLER, 2021):

$$\begin{aligned} \text{hash}(s) &= s[0] + s[1] * p + s[2] * p^2 + \dots + s[n-1] * p^{n-1} \bmod M \\ &= \sum_{i=0}^{n-1} s[i] * p^i \bmod M \end{aligned} \tag{1}$$

- p seria como o valor da base na conversão de números;
- Pode ser um número primo próximo ao número de letras do alfabeto considerado. Por exemplo, para o idioma inglês:
 - $p = 31$ para apenas letras minúsculas;
 - $p = 53$ para letras minúsculas e maiúsculas.

Implementação: hash para strings

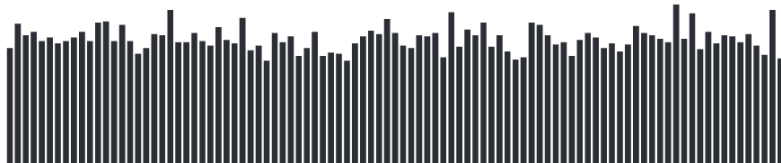
Uma possível implementação:

```
1  int string_hash(char * s) {
2      char c;
3      int p = 31, m = 97;
4      int hash_value = 0, p_pow = 1;
5
6      while (c = *s++){
7          hash_value = (hash_value +
8                      (c - 'a' + 1) * p_pow) % m;
9          p_pow = (p_pow * p) % m;
10     }
11     return(hash_value);
12 }
```

Propriedades importantes de Funções Hash

- Para duas chaves iguais, o valor da função hash deve ser o mesmo;
- O oposto não precisa ser verdadeiro:
 - Se $t \neq s$, não há problemas se $hash(t) = hash(s)$;
 - Embora tente-se evitar ao máximo esse comportamento (evitar colisões);
- Precisa ser calculada de maneira eficiente ($O(1)$ se possível);

Exemplo hash para strings - Implementação do Java



Hash value frequencies for words in *Tale of Two Cities* ($M = 97$)

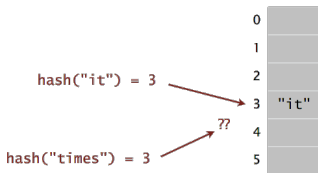
Fonte: (SEdgeWICK; WAYNE, 2014)

Encadeamento Externo (Separate Chaining)

Colisões

Colisão: Duas chaves diferentes recebem o mesmo valor hash.

- Ou temos memória suficiente para evitar esse tipo de problema;
- Ou utilizamos formas para tratar eficientemente eventuais colisões.



Fonte: (SEdgeWICK; WAYNE, 2014)

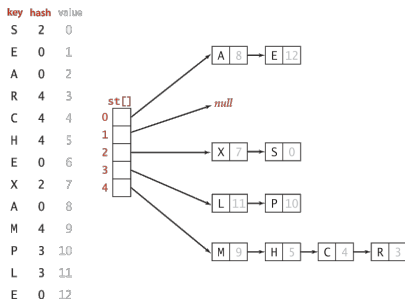
Duas estratégias:

- Encadeamento externo (Separate Chaining);
- Encadeamento interno (Linear Probing).

Encadeamento externo (Separate Chaining)

Utilizar um array de $M < N$ listas ligadas:

- Hash: mapeia uma chave para um inteiro i entre 0 e $M - 1$;
- Inserção: inserir elemento na i -ésima lista (se já não estiver lá);
- Busca: pesquisa **apenas** na i -ésima lista.



Fonte: (SEdgeWICK; WAYNE, 2014)

Análise

Proposição: Supondo uma função hash uniforme, a probabilidade de o número de itens em cada lista ser um fator constante de N/M é muito próximo de 1.

Consequência: Número de operações para busca/inserção é proporcional a N/M :

- M muito grande $\rightarrow$ muitas listas vazias;
- M muito pequeno $\rightarrow$ listas muito longas;
- Escolha típica $\rightarrow M \sim N/4$ (operações em tempo constante).

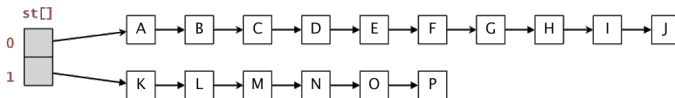
Redimensionamento da tabela

Objetivo: ter listas de tamanho médio N/M (operações em tempo constante).

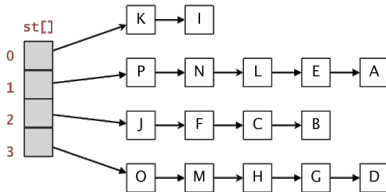
- Duplicar o tamanho da tabela M quando $N/M \geq 8$;
- Reduzir o tamanho da tabela M pela metade quando $N/M \leq 8$;
- É necessário recalcular o hash de todos os elementos quando o tamanho da tabela é alterado. [$\text{hash}(x)$ pode ser diferente para um M diferente]

Redimensionamento da tabela

before resizing



after resizing



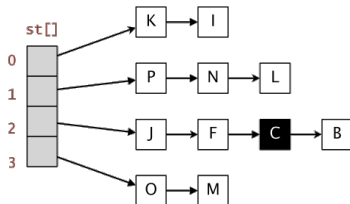
Fonte: (SEDGEWICK; WAYNE, 2014)

Remoção

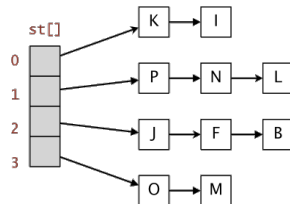
Q: como remover uma chave da tabela?

R: remoção simples da lista que contém o elemento.

before deleting C



after deleting C



Fonte: (SEDGWICK; WAYNE, 2014)

Resumo Implementações Tabela de Símbolos

implementation	guarantee			average case			ordered ops?	key interface
	search	insert	delete	search hit	insert	delete		
sequential search (unordered list)	N	N	N	$\frac{1}{2} N$	N	$\frac{1}{2} N$		<code>equals()</code>
binary search (ordered array)	$\lg N$	N	N	$\lg N$	$\frac{1}{2} N$	$\frac{1}{2} N$	✓	<code>compareTo()</code>
BST	N	N	N	$1.39 \lg N$	$1.39 \lg N$	$\sqrt{N}$	✓	<code>compareTo()</code>
red-black BST	$2 \lg N$	$2 \lg N$	$2 \lg N$	$1.0 \lg N$	$1.0 \lg N$	$1.0 \lg N$	✓	<code>compareTo()</code>
separate chaining	N	N	N	$3-5 *$	$3-5 *$	$3-5 *$		<code>equals()</code> <code>hashCode()</code>

* under uniform hashing assumption

Fonte: (SEdgeWICK; WAYNE, 2014)

Motivação
oooo

Funções Hash
oooooooooooooooo

Encadeamento Externo (Separate Chaining)
oooooooo

Encadeamento Interno (Linear Probing)
●oooooooooooo

Encadeamento Interno (Linear Probing)

Encadeamento Interno (Linear Probing)

Definição: Quando uma nova chave colide, encontre o **próximo** espaço disponível na tabela.

st[0]	jocularly
st[1]	<i>null</i>
st[2]	listen
st[3]	suburban
⋮	<i>null</i>
st[30000]	browsing

linear probing ($M = 30001$, $N = 15000$)

Fonte: (SEdgeWICK; WAYNE, 2014)

Encadeamento Interno (Linear Probing)

- Hash: mapeia uma chave para um inteiro i entre 0 e $M - 1$;
- Inserção: inserir elemento na i -ésima posição da tabela; se ocupado, procurar próxima posição livre;
- Busca: pesquisar na i -ésima posição da tabela; se ocupado mas não encontrado, buscar em $i + 1$, $i + 2$, etc.

Encadeamento Interno (Linear Probing)

- Hash: mapeia uma chave para um inteiro i entre 0 e $M - 1$;
- Inserção: inserir elemento na i -ésima posição da tabela; se ocupado, procurar próxima posição livre;
- Busca: pesquisar na i -ésima posição da tabela; se ocupado mas não encontrado, buscar em $i + 1$, $i + 2$, etc.

Nota: Tamanho de M **deve** ser maior do que o número de chaves N .

Exemplo

key	hash	value	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
S	6	0							S 0									
E	10	1							S 0				E 1					
A	4	2					A 2		S 0				E 1					
R	14	3					A 2		S 0				E 1				R 3	
C	5	4					A 2	C 4	S 0				E 1				R 3	
H	4	5					A 2	C 4	S 0	H 5			E 1				R 3	
E	10	6					A 2	C 4	S 0	H 5			E 6				R 3	
X	15	7					A 2	C 4	S 0	H 5			E 6				R 3	X 7
A	4	8					A 8	C 4	S 0	H 5			E 6				R 3	X 7
M	1	9		M 9			A 8	C 4	S 0	H 5			E 6				R 3	X 7
P	14	10	P 10	M 9			A 8	C 4	S 0	H 5			E 6				R 3	X 7
L	6	11	P 10	M 9			A 8	C 4	S 0	H 5	L 11		E 6				R 3	X 7
E	10	12	P 10	M 9			A 8	C 4	S 0	H 5	L 11		E 12				R 3	X 7

entries in red are new

entries in gray are untouched

keys in black are probes

probe sequence wraps to 0

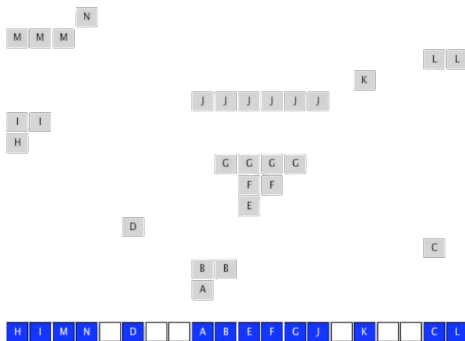
keys[]

vals[]

Fonte: (SEdgeWICK; WAYNE, 2014)

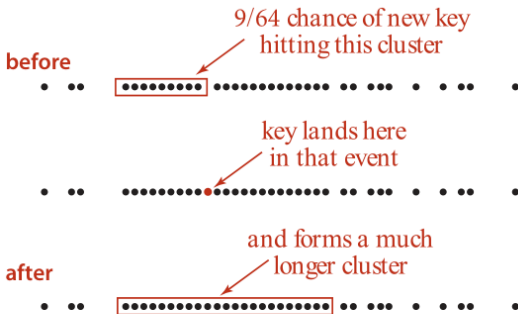
Clustering

Cluster: Um bloco contíguo de itens; **Importante:** Novas chaves tem mais chances de “caírem” em clusters grandes:



Fonte: (SEDGEWICK; WAYNE, 2014)

Clustering: exemplos



Clustering in linear probing ($M = 64$)

Fonte: (SEDGEWICK; WAYNE, 2014)

Clustering: exemplos

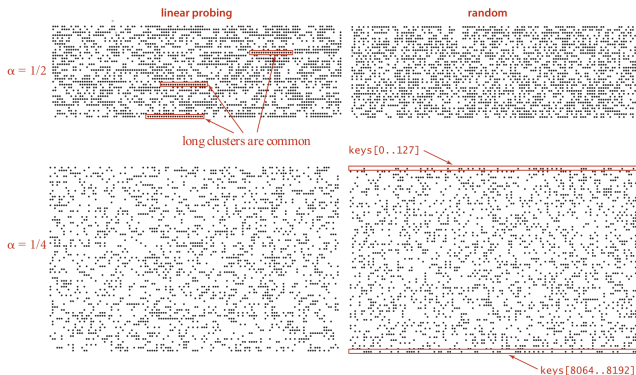


Table occupancy patterns (2,048 keys, tables laid out in 128-position rows)

Fonte: (SEGEWICK; WAYNE, 2014)

$$\alpha = N/M$$

Análise

Proposição: Supondo uma função hash uniforme, o número médio de operações com encadeamento interno para uma tabela de tamanho M que contém $N = \alpha M$ chaves é:

Search hit:

$$\sim \frac{1}{2} \left(1 + \frac{1}{1 - \alpha} \right)$$

Search miss/inserção:

$$\sim \frac{1}{2} \left(1 + \frac{1}{(1 - \alpha)^2} \right)$$

Consequência: Ou seja, constante, mas depende da manutenção da relação $\alpha = N/M$:

- M muito grande $\rightarrow$ muitas posições vazias;
- M muito pequeno $\rightarrow$ tempo de busca linear;
- Escolha típica $\rightarrow \alpha = N/M \sim \frac{1}{2}$ (operações para busca/inserção $\sim 5/2$).

Redimensionamento da tabela

before resizing

	0	1	2	3	4	5	6	7
keys[]		E	S			R	A	
vals[]		1	0			3	2	

after resizing

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
keys[]					A		S				E				R	
vals[]					2		0				1				3	

Fonte: (SEDGWICK; WAYNE, 2014)

Remoção

Q: como remover uma chave da tabela?

R: exige um pouco de cuidado.

before deleting S

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
keys[]	P	M			A	C	S	H	L		E				R	X
vals[]	10	9			8	4	0	5	11		12				3	7

after deleting S ?

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
keys[]	P	M			A	C		H	L		E				R	X
vals[]	10	9			8	4		5	11		12				3	7

doesn't work, e.g., if $\text{hash}(H) = 4$

Fonte: (SEdgeWICK; WAYNE, 2014)

Resumo Implementações Tabela de Símbolos

implementation	guarantee			average case			ordered ops?	key interface
	search	insert	delete	search hit	insert	delete		
sequential search (unordered list)	N	N	N	$\frac{1}{2} N$	N	$\frac{1}{2} N$		<code>equals()</code>
binary search (ordered array)	$\lg N$	N	N	$\lg N$	$\frac{1}{2} N$	$\frac{1}{2} N$	✓	<code>compareTo()</code>
BST	N	N	N	$1.39 \lg N$	$1.39 \lg N$	$\sqrt{N}$	✓	<code>compareTo()</code>
red-black BST	$2 \lg N$	$2 \lg N$	$2 \lg N$	$1.0 \lg N$	$1.0 \lg N$	$1.0 \lg N$	✓	<code>compareTo()</code>
separate chaining	N	N	N	3-5 *	3-5 *	3-5 *		<code>equals()</code> <code>hashCode()</code>
linear probing	N	N	N	3-5 *	3-5 *	3-5 *		<code>equals()</code> <code>hashCode()</code>

* under uniform hashing assumption

Fonte: (SEGEWICK; WAYNE, 2014)

Referências utilizadas na apresentação

 KOGLER, J. *Competitive Programming Algorithms String Hashing*. 2021. Disponível em: <<https://cp-algorithms.com/string/string-hashing.html>>.

 SEDGEWICK, R.; WAYNE, K. *Algorithms: Part I*. [S.I.]: Addison-Wesley Professional, 2014.